

# Clautolisp Design — Host Object Ontology, Lifecycles, Reactors

Codex

June 24, 2026

## Contents

|          |                                                        |          |
|----------|--------------------------------------------------------|----------|
| <b>1</b> | <b>Purpose</b>                                         | <b>1</b> |
| <b>2</b> | <b>Why the ontology comes first</b>                    | <b>2</b> |
| <b>3</b> | <b>The host object graph</b>                           | <b>2</b> |
| <b>4</b> | <b>Lifecycle vocabulary</b>                            | <b>3</b> |
| 4.1      | Maximal lifecycle (the union of all classes) . . . . . | 3        |
| 4.2      | Per-class state machines . . . . .                     | 4        |
| <b>5</b> | <b>The event channel</b>                               | <b>4</b> |
| 5.1      | Event scope rule . . . . .                             | 4        |
| 5.2      | Public emission API . . . . .                          | 5        |
| <b>6</b> | <b>Reactor surface (Phase 14b)</b>                     | <b>6</b> |
| 6.1      | Reactor data model . . . . .                           | 6        |
| 6.2      | Where reactors live . . . . .                          | 6        |
| 6.3      | Persistence . . . . .                                  | 6        |
| 6.4      | Notification modes . . . . .                           | 7        |
| <b>7</b> | <b>Cross-cutting concerns</b>                          | <b>7</b> |
| 7.1      | Vetoing . . . . .                                      | 7        |
| 7.2      | Re-entrancy . . . . .                                  | 7        |
| 7.3      | Snapshot / restore . . . . .                           | 7        |
| <b>8</b> | <b>Testing strategy</b>                                | <b>8</b> |

## 1 Purpose

This document is the architectural blueprint for the AutoLISP / Visual LISP host-object ontology in clautolisp: the typed graph of runtime objects (application, document-manager, document, ...), the lifecycle each one goes through, and the reactor / event channel that fires on lifecycle transitions.

It is the prose companion to:

- autolisp-spec chapter 23 **Host Object Ontology and Lifecycles** — the normative entry that pins the same vocabulary down for the **language**.
- clautolisp/autolisp-runtime/source/ontology.lisp + the runtime-session slots — the structural implementation.
- clautolisp/autolisp-builtins-core’s vlr-\* family — the AutoLISP-visible reactor surface.

The implementation lands in two phases:

- **Phase 14a:** ontology + lifecycle states + the event channel. No reactor surface is exposed yet; backends emit events into a channel that has no subscribers.
- **Phase 14b:** the vlr-\* reactor builtin family on top, with per-document and per-application registries, persistence, and introspection.

## 2 Why the ontology comes first

A reactor only makes sense once the **thing being reacted to** is itself defined. Without an explicit object graph and lifecycle the event vocabulary is a flat list of ‘vlr-foo’ keywords with no classification — it works as a lookup table but degrades quickly once we need to:

- decide where a reactor **lives** (per-document / per-application),
- decide **who is allowed to subscribe** to which event,
- decide **when** a ‘\*WillChange’ precedes a ‘\*Changed’,

- decide whether an event can be **vetoed**,
- serialise **persistent** reactors against the right document.

The ontology answers all five questions in one place. The reactor surface then becomes a thin consumer: a reactor is a tuple (**scope**, **type**, **callback-table**, **owners**) that subscribes to events emitted by the ontology's lifecycle transitions.

### 3 The host object graph

The graph is small and stable; it has been the same shape since AutoCAD R14's Visual LISP merger.

```

Application                                (one per process)
+----- DocumentManager                    (the documents collection)
|   `----- Document (xN)                 (one per open drawing)
|           +----- ModelSpace             (block, kind = :model-space)
|           +----- PaperSpace             (block, kind = :paper-space)
|           +----- Blocks                 (block-record table, xN)
|           +----- Layers                 (xN)
|           +----- Linetypes              (xN)
|           +----- TextStyles             (xN)
|           +----- DimStyles              (xN)
|           +----- Views                  (xN)
|           +----- UCSs                   (xN)
|           +----- Viewports              (xN)
|           +----- RegisteredApplications (xN)
|           +----- NamedObjectsDictionary (root)
|           |   `----- Sub-dictionaries (arbitrary nesting)
|           |           +----- SystemVariables (per-document sysvar bindings)
|           |           +----- ActiveSelectionSet (pickfirst pickset, one)
|           |           +----- Picksets        (user-built xN)
|           |           +----- Entities        (graphical objects xN)
|           |           +----- PendingCommand (active command's state, if any)
|           |           `----- UndoStack      (undo / redo state)
+----- Preferences                        (application-wide)
+----- ARXModules                        (loaded ObjectARX / BRX modules)
+----- LispEnvironment                   (loaded LISP code, separate-VLX namespaces)
+----- ApplicationReactors               (vlr-docmanager / vlr-editor / vlr-linker

```

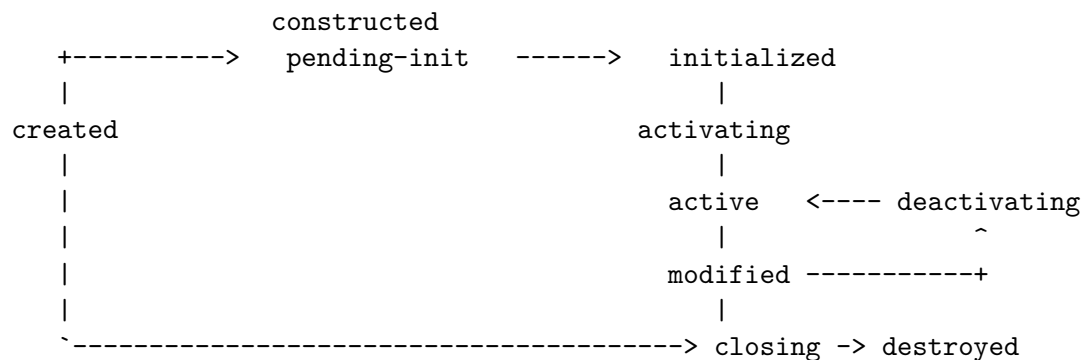
```
|
/ vlr-lisp / vlr-miscellaneous)
`----- PromptHistory / CommandLine (editor I/O)
```

Most of this already has a representation in the runtime (`runtime-session`, `document-namespace`, `mock-host` entity table, `sysvar` table, `picksets`, etc.). Phase 14a adds the missing pieces: the `application` node, the `document-manager` node, lifecycle **state** slots, and the per-class event channel.

## 4 Lifecycle vocabulary

Every class in the graph has a small finite-state machine. State names are deliberately uniform: any host that **adds** a state should be able to use the same vocabulary.

### 4.1 Maximal lifecycle (the union of all classes)



Subclasses of host objects use a subset of these states.

### 4.2 Per-class state machines

| Class            | States                                                               |
|------------------|----------------------------------------------------------------------|
| application      | :running → :quitting → :quit                                         |
| document-manager | always :running                                                      |
| document         | :loading → :loaded <-> :active <-> :inactive → :closing → :closed    |
| command          | :pending → :started → :running → (:ended   :cancelled   :failed)     |
| entity           | :created → :committed <-> :modified → (:erased <-> :unerased) → :p   |
| selection-set    | :created → :populated <-> :mutated → :released                       |
| dictionary-entry | :added → :present → (:renamed <-> :mutated) → :removed               |
| sysvar           | always :bound; transitions are "value will change" / "value changed" |
| xref             | :unresolved → :resolved <-> :unloaded <-> :reloaded → :detached      |
| wblock-op        | :started → (:completed   :cancelled)                                 |
| undo-op          | :begin → (:commit   :rollback)                                       |
| mouse-event      | :down → (:drag*) → :up                                               |
| vlx-app          | :loading → :loaded → :unloaded                                       |
| lisp-eval        | :loading-source → :running → :idle                                   |
| reactor          | :armed <-> :disabled → :removed                                      |

## 5 The event channel

Reactor reactions are systematic names for transitions in the table above. The naming convention has two components:

1. A *\*WillChange* / *\*ToBeFooed* reaction fires **before** the transition; the world is still in the old state. The callback can sometimes veto the transition.
2. A *\*Changed* / *\*BecameFoo* / *\*Fooed* reaction fires **after** the transition; the world is in the new state.

This is why the documented reactor reactions look so verbose — they are the cross-product of the state machine and the WillChange / Changed framing.

### 5.1 Event scope rule

Each event has a single canonical **scope** — application or document — and a single canonical **reactor type**. Examples:

| Event                      | Scope       | Reactor type                                    |
|----------------------------|-------------|-------------------------------------------------|
| :vlr-documentCreated       | application | vlr-docmanager-reactor                          |
| :vlr-documentToBeDestroyed | application | vlr-docmanager-reactor                          |
| :vlr-beginDocumentClose    | document    | vlr-document-reactor                            |
| :vlr-objectModified        | document    | vlr-acdb-reactor or vlr-object-reactor (with ow |
| :vlr-sysVarChanged         | document    | vlr-sysvar-reactor                              |
| :vlr-commandWillStart      | document    | vlr-command-reactor                             |
| :vlr-beginQuit             | application | vlr-editor-reactor                              |
| :vlr-beginARXLoad          | application | vlr-linker-reactor                              |

Some events fire on **both scopes** because both reactor families documented them. Document-activation is the canonical case: `vlr-docmanager-reactor` subscribers see every document's `:vlr-documentBecameCurrent`; `vlr-document-reactor` subscribers on document X see only X's. The runtime's dispatch helper handles this by re-emitting application-scope events that have a document counterpart.

## 5.2 Public emission API

Backends do not implement reactor logic. They **report** events to the runtime. Two public helpers:

```
(signal-document-event document
                          reactor-type
                          reaction-name
                          arguments)

(signal-application-event session
                          reactor-type
                          reaction-name
                          arguments)
```

These helpers walk the appropriate registry and dispatch callbacks under the right document's evaluation context. In Phase 14a they have **no subscribers** — the registries are empty — so emission is observable only via a debug listener used by tests. Phase 14b adds the registries and the AutoLISP-visible `vlr-*` surface that subscribes to them.

## 6 Reactor surface (Phase 14b)

### 6.1 Reactor data model

A reactor is:

```
(defstruct reactor
  (id      (gensym "VLR-") :type t)
  (kind    :object         :type keyword) ; :acdb / :object / :command / ...
  (scope   :document       :type keyword) ; :application / :document
  (callbacks (make-hash-table :test #'eq)
             :documentation "Reaction-name keyword -> AutoLISP function designator
                             (a symbol; not a closure - symbols persist).")
  (owners   '()            :type list)     ; entity / document / pickset handles
  (data     nil)           ; user-supplied opaque list
  (active-p t              :type boolean)  ; toggled by vlr-set-notification
  (persistent-p nil        :type boolean)  ; toggled by vlr-pers
  (document nil)           ; nil for application-scoped reactor
  (notification :all-documents :type keyword)) ; or :current-document-only
```

### 6.2 Where reactors live

| Reactor scope   | Storage                                               |
|-----------------|-------------------------------------------------------|
| Per-document    | document-namespace-reactors — hash-table id → reactor |
| Per-application | runtime-session-application-reactors — same shape     |

The runtime keeps both. `vlr-reactors` enumerates by walking both, subject to its arguments.

### 6.3 Persistence

A persistent reactor (`vlr-pers`) is stored in the **document's** named-object dictionary at a known sub-key (`ACAD_REACTORS` in real AutoCAD; `clautolisp` follows that convention). Storage is by **symbol name**, not by closure: closures can't be serialised across save/reload, but symbol names re-resolve on load.

On document open, the runtime walks `ACAD_REACTORS` and reconstructs each persistent reactor before user code runs. If a callback's symbol is undefined at fire time, dispatch records the failure to the document's prompt-output stream and continues — the reactor itself is not removed.

## 6.4 Notification modes

`vlr-set-notification` flips the `notification` slot:

- `:all-documents` — fire on every document context (default).
- `:current-document-only` — fire only when the active document matches the reactor's owning document.
- `:disabled` — never fire (legacy alias: `vlr-set-notification` with the symbol `'disabled'`).

## 7 Cross-cutting concerns

### 7.1 Vetoing

A few `*WillChange` events let the callback veto the transition. clautolisp models this through the callback's **return value**:

- A non-nil return permits the transition (default).
- The symbol `:veto` rejects it; the runtime aborts the transition and signals a `:transition-vetoed` runtime error to the caller that initiated it.

Only events documented as vetoable by the vendor honour this: `:vlr-documentLockModeWillChange`, `:vlr-aboutToBeDeactivated`, and the `*WillStart` family of command reactions.

### 7.2 Re-entrancy

A callback that itself triggers another lifecycle transition can deliver nested events. The runtime tolerates re-entry but caps the nesting depth at 16 to keep runaway callbacks bounded. Beyond the cap a `:reactor-recursion-limit` runtime error is raised.

### 7.3 Snapshot / restore

The MockHost `mock-host-snapshot` helper already serialises entities, picksets, tables, and sysvars. Phase 14b extends it to serialise persistent reactors via the same NOD path real AutoCAD uses. Non-persistent reactors are **not** snapshotted; they are host-state, not document state.



## 8 Testing strategy

Phase 14a tests:

- The ontology structs construct correctly with default states.
- Lifecycle transitions advance the right state slot.
- `signal-document-event` / `signal-application-event` append events to a debug listener (a test-only side channel).
- `MockHost` mutations emit the right canonical event names.

Phase 14b tests:

- Reactor registration installs in the right registry per scope.
- Each reactor type's callback is invoked when the matching event fires; non-matching events are ignored.
- Veto path: a vetoing callback aborts the transition and the runtime signals `:transition-vetoed`.
- `vlr-set-notification` honours `:current-document-only`.
- Persistence round-trip: register a persistent reactor, snapshot the host, restore from the snapshot, fire the event again, the reactor still triggers.
- Symbol-undefined recovery: register a persistent reactor whose callback symbol does not exist; firing the event records the failure and does not remove the reactor.
- Re-entry cap: a callback that schedules its own event is capped at the configured depth.

## 9 Cross-references

- [autolisp-spec/documentation/autolisp-visual-lisp-specification-draft.org](http://autolisp-spec/documentation/autolisp-visual-lisp-specification-draft.org)  
— chapter 23 **Host Object Ontology and Lifecycles** is the normative companion.
- [clautolisp/PLAN.md + documentation/implementation-roadmap.org](http://clautolisp/PLAN.md+documentation/implementation-roadmap.org)  
— Phase 14a / 14b trackers.

- `clautolisp/autolisp-runtime/source/ontology.lisp` — the runtime data structures.
- `clautolisp/autolisp-mock-host/source/*-api.lisp` — the existing mutation paths that grow `signal-*-event` calls.
- `clautolisp/autolisp-builtins-core/source/api.lisp` — the `vlr-*` AutoLISP-visible surface.